I am working on a Number Guessing Repair Lab using Python and Pygame.

I have a deliberate bug in game/game_engine.py. I want you to fix ONLY Task 1 for now. Do not implement Tasks 2, 3, or 4 yet.

Here is my current game_engine.py:

import random
import pygame

Show more

👍 Here is **Task 1 only**. Nothing related to Tasks 2–4 has been changed.

1. Complete corrected `game/game_engine.py`

Python Run

```python
import random
import pygame
from game.text_box import TextBox


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.secret_number = random.randint(1, 100)
        self.attempts = 0
        self.feedback_msg = "Enter a number between 1 and 100"
        self.feedback_color = (220, 220, 220)
        self.game_won = False

        self.input_box = TextBox(width // 2 - 110, 150, 120, 48)
        self.submit_btn = pygame.Rect(width // 2 + 25, 150, 100, 48)

        self.font_title = pygame.font.SysFont(None, 42)
        self.font_medium = pygame.font.SysFont(None, 28)
        self.font_btn = pygame.font.SysFont(None, 26)

    def submit_guess(self):
        if self.game_won:
```

```
            return

        # Task 1: Validate empty input before converting to integer
        if not self.input_box.text.strip():
            self.feedback_msg = "Please enter a valid number."
            self.feedback_color = (255, 200, 80)
            return

        guess = int(self.input_box.text)

        self.attempts += 1
        self.input_box.clear()

        if guess < self.secret_number:
            self.feedback_msg = f"TOO LOW! (Guess was {guess})"
            self.feedback_color = (80, 160, 240)
        elif guess > self.secret_number:
            self.feedback_msg = f"TOO HIGH! (Guess was {guess})"
            self.feedback_color = (240, 100, 80)
        else:
            self.feedback_msg = f"CORRECT! Found in {self.attempts} attempts."
            self.feedback_color = (80, 220, 90)
            self.game_won = True

    def reset(self):
        self.secret_number = random.randint(1, 100)
        self.attempts = 0
        self.feedback_msg = "Enter a number between 1 and 100"
        self.feedback_color = (220, 220, 220)
        self.game_won = False
        self.input_box.clear()

    def handle_event(self, event):
        self.input_box.handle_event(event)

        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RETURN:
                self.submit_guess()
            elif event.key == pygame.K_r and self.game_won:
                self.reset()

        elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            if self.submit_btn.collidepoint(event.pos):
                self.submit_guess()

    def update(self):
        pass

    def render(self, screen):
        screen.fill((30, 34, 42))

        title_surf = self.font_title.render(
```

```
            "Number Guessing Arena",
            True,
            (245, 245, 245)
        )
        screen.blit(
            title_surf,
            (
                self.width // 2 - title_surf.get_width() // 2,
                35
            )
        )

        attempts_surf = self.font_medium.render(
            f"Attempts: {self.attempts}",
            True,
            (180, 185, 195)
        )
        screen.blit(
            attempts_surf,
            (
                self.width // 2 - attempts_surf.get_width() // 2,
                95
            )
        )

        self.input_box.render(screen)

        pygame.draw.rect(
            screen,
            (50, 150, 80),
            self.submit_btn,
            border_radius=6
        )
        pygame.draw.rect(
            screen,
            (220, 220, 220),
            self.submit_btn,
            width=2,
            border_radius=6
        )

        btn_text = self.font_btn.render(
            "SUBMIT",
            True,
            (255, 255, 255)
        )
        screen.blit(
            btn_text,
            (
                self.submit_btn.centerx - btn_text.get_width() // 2,
                self.submit_btn.centery - btn_text.get_height() // 2
            ),
```

```
            )

            feedback_surf = self.font_medium.render(
                self.feedback_msg,
                True,
                self.feedback_color
            )
            screen.blit(
                feedback_surf,
                (
                    self.width // 2 - feedback_surf.get_width() // 2,
                    235
                )
            )

            if self.game_won:
                restart_surf = self.font_medium.render(
                    "Press [R] to Start a New Game",
                    True,
                    (255, 220, 80)
                )
                screen.blit(
                    restart_surf,
                    (
                        self.width // 2 - restart_surf.get_width() // 2,
                        295
                    )
                )
```

2. Exact change

The original code did this immediately:

Python Run

```
guess = int(self.input_box.text)
```

When the box is empty, `self.input_box.text` is `""`, so Python effectively tries:

Python Run

```
int("")
```

which raises a `ValueError`.

I added this **before** the conversion:

Python Run

```
        if not self.input_box.text.strip():
            self.feedback_msg = "Please enter a valid number."
            self.feedback_color = (255, 200, 80)
            return
```

So now:

- Empty input is detected first.
- `int()` is never called on an empty string.
- `attempts` is **not incremented**.
- The input is left in its current state.
- A warning message is displayed.
- Valid numeric guesses continue to work exactly as before.

3. What to test

After replacing the file and running your game:

Test 1 — Empty input

1. Leave the textbox empty.
2. Click **SUBMIT** or press **Enter**.
3. Expected:
   - Game does **not crash**.
   - Attempts stays at `0`.
   - Feedback says **"Please enter a valid number."**
   - Warning appears in the warning color.

Test 2 — Valid low/high guess

Enter a number such as `10` or `90`.

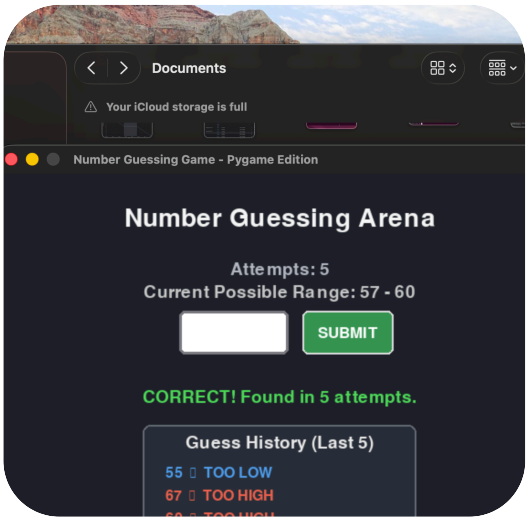
- Attempts should increase by `1`.
- You should get `TOO LOW!` or `TOO HIGH!` depending on the secret number.
- The input box should clear.

Test 3 — Correct guess

Eventually enter the secret number.

- Attempts increases normally.
- `CORRECT! Found in X attempts.` appears.
- The game enters the existing `game_won` state.

This change is limited to **Task 1**; there is no dynamic range, history, maximum-attempt, or `GAME_OVER` logic added.


Documents
Your iCloud storage is full
Number Guessing Game - Pygame Edition
Number Guessing Arena
Attempts: 5
Current Possible Range: 57 - 60
SUBMIT
CORRECT! Found in 5 attempts.
Guess History (Last 5)
55 TOO LOW
67 TOO HIGH

i cant see the last 2 attempts i made

Yes — the history is being recorded. The problem is **the history panel is too low/tall for your current game window**, so the bottom entries are getting cut off.

From your screenshot, I can see the panel starts correctly, but the window reaches the bottom before all 5 rows are visible.

There is also a second small issue: the `↑` and `↓` symbols appear as little boxes on your system because the default Pygame font doesn't support those characters.

Fix

Make the history panel more compact and move it upward slightly. Replace your **Task 3 history section inside** `render()` with this:

Python Run

```python
# Task 3: Guess history panel
history_x = self.width // 2 - 150
history_y = 265
history_width = 300
history_height = 155

pygame.draw.rect(
    screen,
    (42, 47, 57),
    (history_x, history_y, history_width, history_height),
    border_radius=8
)

pygame.draw.rect(
    screen,
    (100, 105, 115),
    (history_x, history_y, history_width, history_height),
    width=2,
    border_radius=8
)

history_title = self.font_medium.render(
    "Guess History (Last 5)",
    True,
    (235, 235, 235)
)
screen.blit(
    history_title,
    (
        self.width // 2 - history_title.get_width() // 2,
```

```
            history_y + 8
        )
    )

    # Display the most recent guesses
    for index, (guess, result, color) in enumerate(self.guess_history):

        # Use ASCII symbols so they display correctly with the default font
        if result == "TOO LOW":
            symbol = "v"
        elif result == "TOO HIGH":
            symbol = "^"
        else:
            symbol = "*"

        history_text = self.font_history.render(
            f"{guess}  {symbol}  {result}",
            True,
            color
        )

        screen.blit(
            history_text,
            (
                history_x + 25,
                history_y + 38 + index * 22
            )
        )

    if self.game_won:
        restart_surf = self.font_medium.render(
            "Press [R] to Start a New Game",
            True,
            (255, 220, 80)
        )
        screen.blit(
            restart_surf,
            (
                self.width // 2 - restart_surf.get_width() // 2,
                history_y + history_height + 12
            )
        )
```

What this changes

The old layout had:

Python Run

```
history_y = 275
history_height = 175
```

and each row was `25` pixels apart.

Now it uses:

Python Run

```python
history_y = 265
history_height = 155
```

and rows are `22` pixels apart.

So all five entries fit much more comfortably:

```
Guess History (Last 5)

55  v  TOO LOW
67  ^  TOO HIGH
60  ^  TOO HIGH
58  v  TOO LOW
59  *  CORRECT
```

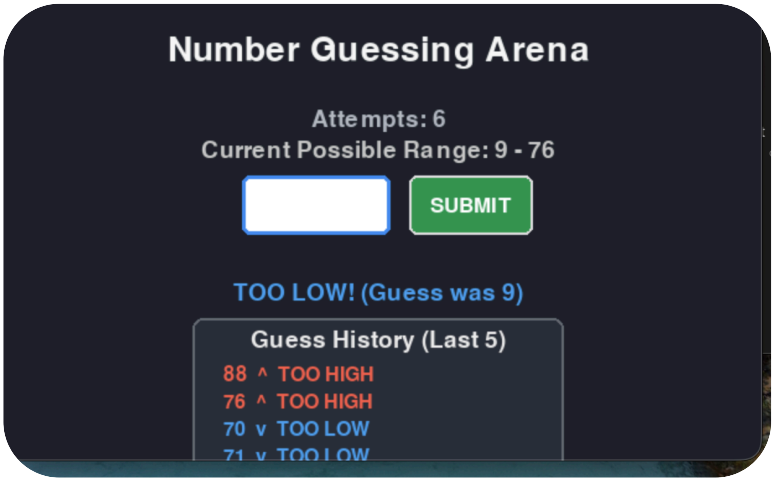
The **history list itself does not need changing**. Your existing:

Python Run

```python
if len(self.guess_history) > 5:
    self.guess_history.pop(0)
```

already ensures that only the latest five valid guesses are stored.

So the issue you're seeing is a **rendering/layout problem, not a history logic problem**.



its correct now

Tasks 1, 2, and 3 are complete and working. Now

implement ONLY Task 4: Maximum Attempts and GAME_OVER.

Modify the current game_engine.py.

Requirements:
- Set a maximum allowance of 7 attempts.
- The attempt count should increase only for valid numeric guesses.

Show more

Yes — for **Task 4**, we add only the 7-attempt limit and `GAME_OVER` state while preserving Tasks 1–3.

Complete final `game_engine.py`

Python Run

```python
import random
import pygame
from game.text_box import TextBox


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.secret_number = random.randint(1, 100)

        # Task 4: Maximum number of valid attempts
        self.max_attempts = 7

        self.attempts = 0

        # Task 2: Dynamic range bounds
        self.low_bound = 1
        self.high_bound = 100

        # Task 3: Guess history
        self.guess_history = []

        self.feedback_msg = "Enter a number between 1 and 100"
        self.feedback_color = (220, 220, 220)

        self.game_won = False

        # Task 4: Game-over state
        self.game_over = False
```

```
        self.input_box = TextBox(width // 2 - 110, 150, 120, 48)
        self.submit_btn = pygame.Rect(width // 2 + 25, 150, 100, 48)

        self.font_title = pygame.font.SysFont(None, 42)
        self.font_medium = pygame.font.SysFont(None, 28)
        self.font_btn = pygame.font.SysFont(None, 26)
        self.font_history = pygame.font.SysFont(None, 24)

    def submit_guess(self):
        # Do not accept guesses after winning or game over
        if self.game_won or self.game_over:
            return

        # Task 1: Validate empty input before converting to integer
        if not self.input_box.text.strip():
            self.feedback_msg = "Please enter a valid number."
            self.feedback_color = (255, 200, 80)
            return

        guess = int(self.input_box.text)

        # Only valid numeric guesses increase attempts
        self.attempts += 1
        self.input_box.clear()

        if guess < self.secret_number:
            # Task 2: Update lower boundary
            self.low_bound = guess

            self.feedback_msg = f"TOO LOW! (Guess was {guess})"
            self.feedback_color = (80, 160, 240)

            # Task 3: Add guess to history
            self.guess_history.append(
                (guess, "TOO LOW", (80, 160, 240))
            )

        elif guess > self.secret_number:
            # Task 2: Update upper boundary
            self.high_bound = guess

            self.feedback_msg = f"TOO HIGH! (Guess was {guess})"
            self.feedback_color = (240, 100, 80)

            # Task 3: Add guess to history
            self.guess_history.append(
                (guess, "TOO HIGH", (240, 100, 80))
            )

        else:
            # Existing victory behavior
```

```
                self.feedback_msg = (
                    f"CORRECT! Found in {self.attempts} attempts."
                )
                self.feedback_color = (80, 220, 90)
                self.game_won = True

                # Task 3: Record correct guess
                self.guess_history.append(
                    (guess, "CORRECT", (80, 220, 90))
                )

            # Task 3: Keep only the most recent 5 guesses
            if len(self.guess_history) > 5:
                self.guess_history.pop(0)

            # Task 4: Check maximum attempts
            if not self.game_won and self.attempts >= self.max_attempts:
                self.game_over = True

                self.feedback_msg = (
                    f"GAME OVER! The secret number was {self.secret_number}."
                )
                self.feedback_color = (240, 100, 100)

        def reset(self):
            # Start a completely new game
            self.secret_number = random.randint(1, 100)
            self.attempts = 0

            # Task 2: Reset dynamic range
            self.low_bound = 1
            self.high_bound = 100

            # Task 3: Reset guess history
            self.guess_history.clear()

            self.feedback_msg = "Enter a number between 1 and 100"
            self.feedback_color = (220, 220, 220)

            self.game_won = False

            # Task 4: Reset game-over state
            self.game_over = False

            self.input_box.clear()

        def handle_event(self, event):
            self.input_box.handle_event(event)

            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_RETURN:
                    self.submit_guess()
```

```
            elif event.key == pygame.K_r and (
                self.game_won or self.game_over
            ):
                self.reset()

        elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            if self.submit_btn.collidepoint(event.pos):
                self.submit_guess()

    def update(self):
        pass

    def render(self, screen):
        screen.fill((30, 34, 42))

        title_surf = self.font_title.render(
            "Number Guessing Arena",
            True,
            (245, 245, 245)
        )
        screen.blit(
            title_surf,
            (
                self.width // 2 - title_surf.get_width() // 2,
                35
            )
        )

        attempts_surf = self.font_medium.render(
            f"Attempts: {self.attempts}/{self.max_attempts}",
            True,
            (180, 185, 195)
        )
        screen.blit(
            attempts_surf,
            (
                self.width // 2 - attempts_surf.get_width() // 2,
                95
            )
        )

        # Task 2: Display current possible range
        range_surf = self.font_medium.render(
            f"Current Possible Range: "
            f"{self.low_bound} - {self.high_bound}",
            True,
            (200, 200, 200)
        )
        screen.blit(
            range_surf,
            (
```

```
            self.width // 2 - range_surf.get_width() // 2,
            120
        )
    )

    self.input_box.render(screen)

    pygame.draw.rect(
        screen,
        (50, 150, 80),
        self.submit_btn,
        border_radius=6
    )
    pygame.draw.rect(
        screen,
        (220, 220, 220),
        self.submit_btn,
        width=2,
        border_radius=6
    )

    btn_text = self.font_btn.render(
        "SUBMIT",
        True,
        (255, 255, 255)
    )
    screen.blit(
        btn_text,
        (
            self.submit_btn.centerx - btn_text.get_width() // 2,
            self.submit_btn.centery - btn_text.get_height() // 2
        ),
    )

    # Feedback message
    feedback_surf = self.font_medium.render(
        self.feedback_msg,
        True,
        self.feedback_color
    )
    screen.blit(
        feedback_surf,
        (
            self.width // 2 - feedback_surf.get_width() // 2,
            235
        )
    )

    # Task 3: Guess history panel
    history_x = self.width // 2 - 150
    history_y = 265
    history_width = 300
```

```
            history_height = 155

            pygame.draw.rect(
                screen,
                (42, 47, 57),
                (history_x, history_y, history_width, history_height),
                border_radius=8
            )

            pygame.draw.rect(
                screen,
                (100, 105, 115),
                (history_x, history_y, history_width, history_height),
                width=2,
                border_radius=8
            )

            history_title = self.font_medium.render(
                "Guess History (Last 5)",
                True,
                (235, 235, 235)
            )
            screen.blit(
                history_title,
                (
                    self.width // 2 - history_title.get_width() // 2,
                    history_y + 8
                )
            )

            # Display the most recent guesses
            for index, (guess, result, color) in enumerate(
                self.guess_history
            ):
                # ASCII symbols to avoid font compatibility issues
                if result == "TOO LOW":
                    symbol = "v"
                elif result == "TOO HIGH":
                    symbol = "^"
                else:
                    symbol = "*"

                history_text = self.font_history.render(
                    f"{guess}  {symbol}  {result}",
                    True,
                    color
                )

                screen.blit(
                    history_text,
                    (
                        history_x + 25,
```

```
                history_y + 38 + index * 22
            )
        )

    # Task 4: End-game message/prompt
    if self.game_won:
        restart_surf = self.font_medium.render(
            "Press [R] to Start a New Game",
            True,
            (255, 220, 80)
        )
        screen.blit(
            restart_surf,
            (
                self.width // 2 - restart_surf.get_width() // 2,
                history_y + history_height + 12
            )
        )

    elif self.game_over:
        game_over_surf = self.font_medium.render(
            "GAME OVER!",
            True,
            (240, 100, 100)
        )
        screen.blit(
            game_over_surf,
            (
                self.width // 2 - game_over_surf.get_width() // 2,
                history_y + history_height + 8
            )
        )

        reveal_surf = self.font_medium.render(
            f"The secret number was {self.secret_number}.",
            True,
            (240, 220, 220)
        )
        screen.blit(
            reveal_surf,
            (
                self.width // 2 - reveal_surf.get_width() // 2,
                history_y + history_height + 38
            )
        )

        restart_surf = self.font_medium.render(
            "Press [R] to Start a New Game",
            True,
            (255, 220, 80)
        )
        screen.blit(
```

```
                restart_surf,
                (
                    self.width // 2 - restart_surf.get_width() // 2,
                    history_y + history_height + 68
                )
            )
```

What was added for Task 4

The maximum is stored as:

Python Run

```
self.max_attempts = 7
```

A new state tracks whether the player has run out of attempts:

Python Run

```
self.game_over = False
```

The important logic is after each valid guess:

Python Run

```
if not self.game_won and self.attempts >= self.max_attempts:
    self.game_over = True
```

So the sequence is:

```
Valid guess → attempts + 1
             ↓
        Correct?
       /        \
     YES         NO
      ↓           ↓
   WIN       attempts == 7?
                  ↓
              GAME OVER
```

Once `game_over` becomes `True`, this prevents additional guesses:

Python Run

```
if self.game_won or self.game_over:
    return
```

The secret number is revealed through:

Python Run

```
self.feedback_msg = (
    f"GAME OVER! The secret number was {self.secret_number}."
)
```

and the UI also displays the GAME OVER state and **Press [R] to Start a New Game**.

Reset behavior

Pressing **R** after either winning or losing now calls `reset()`:

Python Run

```
elif event.key == pygame.K_r and (
    self.game_won or self.game_over
):
    self.reset()
```

`reset()` restores the secret number, attempts, range, history, feedback, `game_won`, and `game_over`.

What to test

Test these three cases:

1. Win before attempt 7:
Guess the correct number on, say, attempt 4. You should get the normal `CORRECT!` behavior, and the game should not become `GAME_OVER`.

2. Lose on attempt 7:
Make seven incorrect valid guesses. On the seventh, you should see:

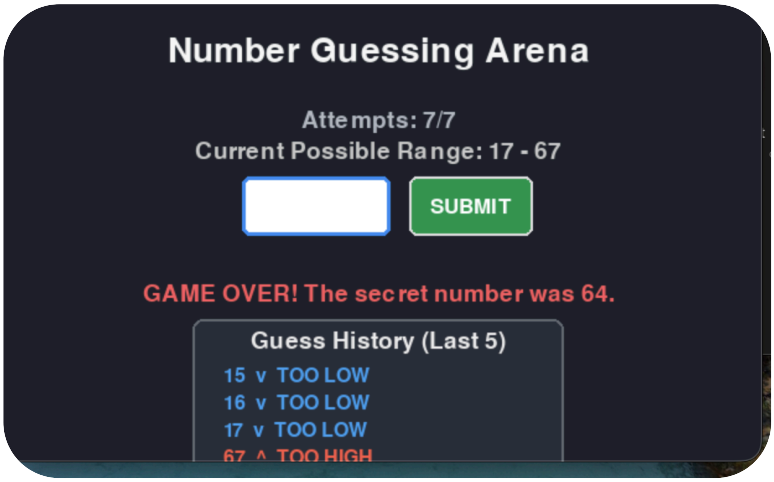
```
GAME OVER!
The secret number was XX.
Press [R] to Start a New Game
```

Attempts should show:

```
Attempts: 7/7
```

Trying to submit an eighth guess should do nothing.

3. Empty input:
Submit an empty box at any point before GAME_OVER. Attempts must **not** increase, preserving Task 1.



there is no prompt to press r minimize the guess history window and allow the press r prompt to be above the GUESS WAS TOO LOW area

Yes — the logic is working. The problem is purely the **layout**: the restart prompt was being drawn *after* the history panel, so it was pushed below the visible game window.

I would make the history panel smaller and put the **R prompt directly below the input controls and above the feedback message**.

Replace your current `game_engine.py` with this version:

Python Run

```python
import random
import pygame
from game.text_box import TextBox


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.secret_number = random.randint(1, 100)

        # Task 4: Maximum attempts
        self.max_attempts = 7

        self.attempts = 0

        # Task 2: Dynamic range
        self.low_bound = 1
```

```
        self.high_bound = 100

        # Task 3: Guess history
        self.guess_history = []

        self.feedback_msg = "Enter a number between 1 and 100"
        self.feedback_color = (220, 220, 220)

        self.game_won = False

        # Task 4: Game over state
        self.game_over = False

        self.input_box = TextBox(width // 2 - 110, 150, 120, 48)
        self.submit_btn = pygame.Rect(width // 2 + 25, 150, 100, 48)

        self.font_title = pygame.font.SysFont(None, 42)
        self.font_medium = pygame.font.SysFont(None, 28)
        self.font_btn = pygame.font.SysFont(None, 26)
        self.font_history = pygame.font.SysFont(None, 22)

    def submit_guess(self):
        # Do not accept guesses after winning or game over
        if self.game_won or self.game_over:
            return

        # Task 1: Validate empty input
        if not self.input_box.text.strip():
            self.feedback_msg = "Please enter a valid number."
            self.feedback_color = (255, 200, 80)
            return

        guess = int(self.input_box.text)

        # Only valid guesses increase attempts
        self.attempts += 1
        self.input_box.clear()

        if guess < self.secret_number:
            # Task 2: Update lower boundary
            self.low_bound = guess

            self.feedback_msg = f"TOO LOW! (Guess was {guess})"
            self.feedback_color = (80, 160, 240)

            # Task 3: Add to history
            self.guess_history.append(
                (guess, "TOO LOW", (80, 160, 240))
            )

        elif guess > self.secret_number:
            # Task 2: Update upper boundary
```

```
            self.high_bound = guess

            self.feedback_msg = f"TOO HIGH! (Guess was {guess})"
            self.feedback_color = (240, 100, 80)

            # Task 3: Add to history
            self.guess_history.append(
                (guess, "TOO HIGH", (240, 100, 80))
            )

        else:
            self.feedback_msg = (
                f"CORRECT! Found in {self.attempts} attempts."
            )
            self.feedback_color = (80, 220, 90)
            self.game_won = True

            # Task 3: Record correct guess
            self.guess_history.append(
                (guess, "CORRECT", (80, 220, 90))
            )

        # Keep only the latest 5 guesses
        if len(self.guess_history) > 5:
            self.guess_history.pop(0)

        # Task 4: Trigger GAME OVER after 7 incorrect guesses
        if not self.game_won and self.attempts >= self.max_attempts:
            self.game_over = True

            self.feedback_msg = (
                f"GAME OVER! The secret number was {self.secret_number}."
            )
            self.feedback_color = (240, 100, 100)

    def reset(self):
        self.secret_number = random.randint(1, 100)
        self.attempts = 0

        # Task 2: Reset range
        self.low_bound = 1
        self.high_bound = 100

        # Task 3: Reset history
        self.guess_history.clear()

        self.feedback_msg = "Enter a number between 1 and 100"
        self.feedback_color = (220, 220, 220)

        self.game_won = False

        # Task 4: Reset game over
```

```
        self.game_over = False

        self.input_box.clear()

    def handle_event(self, event):
        self.input_box.handle_event(event)

        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RETURN:
                self.submit_guess()

            elif event.key == pygame.K_r and (
                self.game_won or self.game_over
            ):
                self.reset()

        elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            if self.submit_btn.collidepoint(event.pos):
                self.submit_guess()

    def update(self):
        pass

    def render(self, screen):
        screen.fill((30, 34, 42))

        # Title
        title_surf = self.font_title.render(
            "Number Guessing Arena",
            True,
            (245, 245, 245)
        )
        screen.blit(
            title_surf,
            (
                self.width // 2 - title_surf.get_width() // 2,
                35
            )
        )

        # Attempts
        attempts_surf = self.font_medium.render(
            f"Attempts: {self.attempts}/{self.max_attempts}",
            True,
            (180, 185, 195)
        )
        screen.blit(
            attempts_surf,
            (
                self.width // 2 - attempts_surf.get_width() // 2,
                95
            )
```

```
        )

        # Dynamic range
        range_surf = self.font_medium.render(
            f"Current Possible Range: "
            f"{self.low_bound} - {self.high_bound}",
            True,
            (200, 200, 200)
        )
        screen.blit(
            range_surf,
            (
                self.width // 2 - range_surf.get_width() // 2,
                120
            )
        )

        # Input box
        self.input_box.render(screen)

        # Submit button
        pygame.draw.rect(
            screen,
            (50, 150, 80),
            self.submit_btn,
            border_radius=6
        )

        pygame.draw.rect(
            screen,
            (220, 220, 220),
            self.submit_btn,
            width=2,
            border_radius=6
        )

        btn_text = self.font_btn.render(
            "SUBMIT",
            True,
            (255, 255, 255)
        )

        screen.blit(
            btn_text,
            (
                self.submit_btn.centerx - btn_text.get_width() // 2,
                self.submit_btn.centery - btn_text.get_height() // 2
            ),
        )

        # ------------------------------------------------
        # Task 4: Restart prompt ABOVE feedback message
```

```
        # ------------------------------------------------
        if self.game_won or self.game_over:
            restart_surf = self.font_medium.render(
                "Press [R] to Start a New Game",
                True,
                (255, 220, 80)
            )

            screen.blit(
                restart_surf,
                (
                    self.width // 2 - restart_surf.get_width() // 2,
                    205
                )
            )

        # Feedback message
        feedback_surf = self.font_medium.render(
            self.feedback_msg,
            True,
            self.feedback_color
        )

        screen.blit(
            feedback_surf,
            (
                self.width // 2 - feedback_surf.get_width() // 2,
                235
            )
        )

        # ------------------------------------------------
        # Task 3: Smaller Guess History panel
        # ------------------------------------------------
        history_x = self.width // 2 - 135
        history_y = 270
        history_width = 270
        history_height = 135

        pygame.draw.rect(
            screen,
            (42, 47, 57),
            (
                history_x,
                history_y,
                history_width,
                history_height
            ),
            border_radius=8
        )

        pygame.draw.rect(
```

```
            screen,
            (100, 105, 115),
            (
                history_x,
                history_y,
                history_width,
                history_height
            ),
            width=2,
            border_radius=8
        )

        # History title
        history_title = self.font_history.render(
            "Guess History (Last 5)",
            True,
            (235, 235, 235)
        )

        screen.blit(
            history_title,
            (
                self.width // 2 - history_title.get_width() // 2,
                history_y + 7
            )
        )

        # History entries
        for index, (guess, result, color) in enumerate(
            self.guess_history
        ):

            if result == "TOO LOW":
                symbol = "v"
            elif result == "TOO HIGH":
                symbol = "^"
            else:
                symbol = "*"

            history_text = self.font_history.render(
                f"{guess}  {symbol}  {result}",
                True,
                color
            )

            screen.blit(
                history_text,
                (
                    history_x + 25,
                    history_y + 34 + index * 19
                )
            )
```

```
        # GAME OVER extra information
        if self.game_over:
            reveal_surf = self.font_history.render(
                f"Secret number: {self.secret_number}",
                True,
                (240, 220, 220)
            )

            screen.blit(
                reveal_surf,
                (
                    self.width // 2 - reveal_surf.get_width() // 2,
                    history_y + history_height + 8
                )
            )
```

What I changed

The important layout change is that the restart prompt is now drawn at:

Python Run

```
205
```

while the feedback remains at:

Python Run

```
235
```

So when you lose, the screen will look roughly like:

```
Attempts: 7/7
Current Possible Range: 17 - 67

[ INPUT ]   [ SUBMIT ]

Press [R] to Start a New Game

GAME OVER! The secret number was 64.

┌──────────────────────────┐
│   Guess History (Last 5) │
│ 15  v  TOO LOW           │
│ 16  v  TOO LOW           │
│ 17  v  TOO LOW           │
│ 67  ^  TOO HIGH          │
```

```
| 64  *  CORRECT          |
└─────────────────────────┘
```

The history panel is also reduced from **300 × 155** to **270 × 135**, with tighter row spacing, so the entire panel fits better within your window.

One important detail: **the game logic has not been changed** here. Tasks 1–4 still work exactly as before; this update mainly fixes the Task 4 screen layout.



ChatGPT can make mistakes. Check important info.